

Module 8: Inheritance in Java

Java Laboratory – Scenario-Based Problem Statements

Module Overview

Inheritance is one of the fundamental pillars of Object-Oriented Programming (OOP). It enables developers to build software systems by reusing existing code, extending functionality, and creating hierarchical relationships between classes. Modern enterprise applications such as banking systems, hospital management systems, e-commerce platforms, ERP software, and educational management systems extensively use inheritance to model real-world relationships.

This module introduces students to various types of inheritance supported by Java, including **Single Inheritance**, **Multilevel Inheritance**, and **Hierarchical Inheritance**. Students will also learn about **method overriding**, the **super** keyword, and **constructor chaining** through practical, scenario-based laboratory exercises.

The exercises are designed to help students understand "**IS-A**" **relationships**, code reusability, extensibility, and object-oriented design principles.

Total Practice Problems: 20

Difficulty Distribution

- **Beginner:** 6 Problems
- **Intermediate:** 8 Problems
- **Advanced:** 6 Problems

Learning Objectives

After completing this module, students will be able to:

- Understand inheritance and its advantages.
- Design parent-child class relationships.
- Implement single, multilevel, and hierarchical inheritance.
- Use the `super` keyword effectively.
- Perform constructor chaining.
- Override inherited methods.
- Model real-world systems using inheritance.

- Promote code reuse through inheritance.
-

Concepts Covered

- Single Inheritance
 - Multilevel Inheritance
 - Hierarchical Inheritance
 - IS-A Relationship
 - Method Overriding
 - `super` Keyword
 - Constructor Chaining
 - Runtime Method Invocation
-

Beginner Level Problems

Problem 1: University Employee Management

Scenario

A university maintains information about different types of employees.

Problem Statement

Create a parent class **Employee** containing:

- Employee ID
- Name
- Department

Create a child class **Faculty** that additionally stores:

- Subject
- Designation
- Experience

Display complete employee information using inheritance.

Concepts Covered

- Single Inheritance
- Constructors

Problem 2: Banking Customer Accounts

Scenario

A bank offers different account types.

Problem Statement

Create a parent class **Account** containing:

- Account Number
- Customer Name
- Balance

Create a child class **SavingsAccount** that includes:

- Interest Rate
- Minimum Balance

Display complete account information.

Problem 3: Vehicle Registration System

Scenario

A transport department manages different vehicles.

Problem Statement

Create a parent class **Vehicle** containing:

- Registration Number
- Manufacturer
- Model

Create a child class **Car** that additionally contains:

- Number of Doors
- Fuel Type

Display vehicle details.

Problem 4: Hospital Patient Management

Scenario

A hospital records patient information.

Problem Statement

Create a parent class **Person**.

Create a child class **Patient** containing:

- Disease
- Ward Number
- Doctor Assigned

Generate a patient report.

Problem 5: Library Book Management

Scenario

A library stores different types of books.

Problem Statement

Create a parent class **Book**.

Create a child class **ReferenceBook** that stores:

- Edition
- Publisher
- Library Section

Display complete book information.

Problem 6: Mobile Store Inventory

Scenario

A mobile showroom sells different electronic devices.

Problem Statement

Create a parent class **ElectronicDevice**.

Create a child class **MobilePhone**.

Display inherited and specialized attributes.

Intermediate Level Problems

Problem 7: University Academic Hierarchy

Scenario

A university categorizes its academic members.

Problem Statement

Create the following hierarchy:

```
Person
  ↓
Employee
  ↓
Faculty
```

Implement multilevel inheritance.

Display complete faculty information.

Problem 8: Banking Loan Processing

Scenario

A bank processes different loan types.

Problem Statement

Create the following hierarchy:

```
Loan
  ↓
HomeLoan
  ↓
PremiumHomeLoan
```

Each class should introduce additional loan-related information.

Use constructor chaining.

Problem 9: Hospital Management System

Scenario

A hospital maintains different categories of staff.

Problem Statement

Create:

```
Person
  ↓
Doctor

Person
  ↓
Nurse

Person
  ↓
Receptionist
```

Implement hierarchical inheritance.

Display role-specific information.

Problem 10: Educational Institution

Scenario

A college manages different categories of students.

Problem Statement

Create:

```
Student
  ↓
UndergraduateStudent

Student
  ↓
PostgraduateStudent

Student
  ↓
ResearchScholar
```

Implement hierarchical inheritance.

Problem 11: Restaurant Management

Scenario

A restaurant employs different categories of workers.

Problem Statement

Create:

```
Employee
  ↓
Chef

Employee
  ↓
Cashier

Employee
  ↓
Waiter
```

Override the method:

```
performDuty()
```

Each subclass should implement its own behavior.

Problem 12: E-Commerce Product Catalog

Scenario

An online shopping platform categorizes products.

Problem Statement

Create:

```
Product
  ↓
Electronics
```

```
Product
  ↓
Clothing
```

```
Product
  ↓
Furniture
```

Override the method:

```
calculateDiscount()
```

Each product category follows different discount rules.

Problem 13: Insurance Management System

Scenario

An insurance company offers multiple insurance plans.

Problem Statement

Create:


```
InsurancePolicy
  ↓
LifeInsurance

InsurancePolicy
  ↓
VehicleInsurance

InsurancePolicy
  ↓
HealthInsurance
```

Override:

```
calculatePremium()
```

Problem 14: Constructor Chaining

Scenario

A software company initializes employee information in multiple stages.

Problem Statement

Create:

```
Person
  ↓
Employee
  ↓
SoftwareEngineer
```

Use the `super` keyword to invoke parent constructors.

Display constructor execution order.

Advanced Level Problems

Problem 15: Smart Banking System

Scenario

A bank offers multiple account services.

Problem Statement

Design the hierarchy:

```
Customer
  ↓
Account
  ↓
SavingsAccount

Account
  ↓
CurrentAccount
```

Override methods:

- deposit()
- withdraw()
- calculateInterest()

Generate an account statement.

Problem 16: Airline Reservation System

Scenario

An airline manages different ticket categories.

Problem Statement

Design:

```
Ticket
  ↓
EconomyTicket

Ticket
  ↓
BusinessTicket

Ticket
  ↓
FirstClassTicket
```

Override:

```
calculateFare()
```

Display ticket summary.

Problem 17: Hospital Information System

Scenario

A hospital wants to manage different departments.

Problem Statement

Design:

HospitalStaff

↓

Doctor

HospitalStaff

↓

Nurse

HospitalStaff

↓

Pharmacist

HospitalStaff

↓

LabTechnician

Each class should override:

```
performDuty()
```

Problem 18: Online Learning Platform

Scenario

An online education platform supports different users.

Problem Statement

Design:

User

↓
Student

User
↓
Instructor

User
↓
Administrator

Override methods:

- login()
- dashboard()
- performRole()

Problem 19: Smart City Transportation System

Scenario

A city transportation department manages various transport services.

Problem Statement

Create:

Transport
↓
Bus

Transport
↓
Metro

Transport
↓
Taxi

Override methods:

- calculateFare()
- displayRoute()

Generate transport reports.

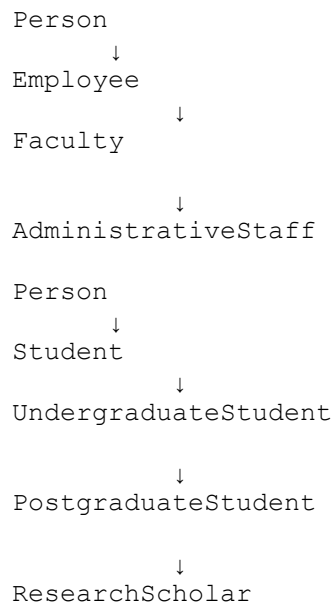
Problem 20: College ERP System

Scenario

A university is developing a College ERP application.

Problem Statement

Design the following hierarchy:



Implement:

- Constructor chaining
- Method overriding
- `super` keyword
- Common and specialized methods

Generate detailed reports for every category.

Challenge Problems

1. Develop a **Hospital Management System** using inheritance with different staff and patient categories.
2. Create a **Banking Application** supporting Savings, Current, Salary, and Fixed Deposit accounts.

3. Design an **Online Shopping Platform** with multiple product categories and discount strategies.
 4. Build an **Airline Reservation System** featuring Economy, Business, and First-Class tickets.
 5. Develop a **College ERP System** managing students, faculty, staff, and administrators.
 6. Create a **Vehicle Rental System** supporting cars, bikes, buses, and trucks with customized rental calculations.
-

Real-World Applications

Students will understand how inheritance is applied in:

- Banking and Financial Systems
 - Hospital Management Systems
 - University ERP Software
 - Airline Reservation Systems
 - E-Commerce Platforms
 - Human Resource Management Systems
 - Government Citizen Portals
 - Insurance Management Applications
 - Logistics and Transportation Systems
 - Enterprise Business Applications
-

Common Mistakes

Students should avoid:

- Choosing inheritance when an "IS-A" relationship does not exist.
 - Forgetting to call parent constructors using `super()`.
 - Redefining inherited fields unnecessarily.
 - Misunderstanding method overriding rules.
 - Omitting the `@Override` annotation.
 - Confusing method overriding with method overloading.
 - Violating encapsulation by directly accessing private parent members.
-

Viva Questions

1. What is inheritance in Java?
 2. What are the advantages of inheritance?
 3. Explain the **IS-A** relationship with an example.
 4. Which types of inheritance are supported by Java classes?
 5. Why does Java not support multiple inheritance through classes?
 6. What is the purpose of the `super` keyword?
 7. Explain constructor chaining.
 8. What is method overriding? State its rules.
 9. Differentiate between inheritance and composition.
 10. When should inheritance be preferred over object composition, and vice versa?
-

Expected Learning Outcomes

After completing this module, students will be able to:

- Design reusable class hierarchies using inheritance.
- Model real-world entities through parent-child relationships.
- Apply single, multilevel, and hierarchical inheritance appropriately.
- Use constructor chaining and the `super` keyword effectively.
- Implement method overriding to provide specialized behavior.
- Develop scalable and maintainable object-oriented applications following industry best practices.

Special Notes

- Before coding, ask students to draw a **UML class diagram** identifying the **IS-A** relationships.
- Reinforce that inheritance should only be used when a true inheritance relationship exists (e.g., *SavingsAccount is an Account*, *Doctor is a HospitalStaff*).
- Demonstrate the flow of constructor execution during object creation using constructor chaining.
- Encourage students to use the `@Override` annotation for all overridden methods and discuss its benefits.
- Conclude the module by comparing **inheritance** with **composition**, preparing students for more advanced object-oriented design principles and design patterns.